Who Wrote It? Identifying LLMs from Their Responses

CMPSC 448 Midterm Project

Junbo Zhao (team leader)
jzz5658@psu.edu

Canyi Cheng
cvc6413@psu.edu

Yehe Cui
yfc5461@psu.edu

The Pennsylvania State University
October 2026

Abstract

We try to figure out which large language model wrote a given chatbot answer. Using 11,915 prompt and response pairs from the Chatbot Arena dataset, balanced across five model families (Claude, GPT, Koala, PaLM and Vicuna), we train a word-level CNN and a bidirectional LSTM from scratch. From the response alone, both models reach about 61% test accuracy, where random guessing gets 20%. The prompt alone is useless (about 21%) because Arena assigns models to prompts at random, and adding the prompt to the response actually hurts a little. Claude and PaLM are easy to spot, while GPT, Vicuna and Koala get mixed up a lot, which makes sense because Vicuna and Koala were both fine-tuned on ChatGPT conversations. For the two extra credit questions, we find that the fingerprint mostly survives a change of task, and that a CNN that sees only punctuation and line breaks, with every word deleted, still gets 56%.

1 Problem Definition

A lot of the text people read online now comes out of a chatbot, and usually nobody tells you which one. The question we look at in this project is whether different large language models leave a recognizable "fingerprint" in the way they answer, and whether a fairly small neural network can pick up on it.

We set this up as a 5-way text classification problem. Every example in our data is a triple (y, x_{in}, x_{out}) where y is the model family that answered, x_{in} is the user's prompt and x_{out} is the model's response. The five classes are Claude, GPT, Koala, PaLM and Vicuna. A classifier is a function f that takes some text and outputs one of the five labels, and depending on the experiment that text is x_{out}, x_{in}, or both glued together.

The project asks two required questions and two optional ones. We did all four:

- RQ1: Can a CNN and an RNN tell which LLM wrote a response, using only x_{out}?
- RQ2: Does the user's prompt help? We train on x_{in} only, x_{out} only, and both.
- RQ3 (extra credit): If the classifier never saw a certain kind of task during training, does the fingerprint still work on it?
- RQ4 (extra credit): What actually makes the models different, and which signals is the classifier relying on?

How we measure it. We report two numbers for every experiment. Accuracy is the fraction of test examples labeled correctly. Since we balance the classes, a classifier that guesses at random gets 20%, and that is the number everything below should be compared against. We also report macro F1. For each class c we compute

$$F1_c = \frac{2\,P_c\,R_c}{P_c + R_c},$$

where P_c (precision) is how often a prediction of c is right and R_c (recall) is how many of the real c examples we found. Macro F1 is the plain average of $F1_c$ over the five classes. We like it because a model that dumps everything into one class can still get a decent accuracy, but its macro F1 will look bad. When accuracy and macro F1 disagree a lot, it usually means one class is doing something weird, and we point that out when it happens.

All code is in the same GitHub repository as this PDF. Appendix A lists the exact commands, and every number in this report comes from `results/summary.txt`, which the scripts write automatically.

2 Dataset

2.1 Where the data comes from

We used the `lmsys/chatbot_arena_conversations` dataset on HuggingFace, released by LMSYS together with the Chatbot Arena paper [1]. It contains 33,000 "battles" collected on the Chatbot Arena website in 2023. In a battle a user types a prompt, two anonymous models answer it side by side, and the user votes for the better answer. For every battle the dataset stores the names of both models and both full conversations, which gives us exactly the `(LLM_name, LLM_input, LLM_output)` triple the project asks for.

We picked this dataset over collecting our own for a few reasons. It is public and the terms are clear (the dataset is gated, so we had to log in and accept its terms of use before downloading). It has real prompts from real users instead of prompts we made up ourselves. And it has one property that turned out to matter a lot for RQ2: the two models in a battle are picked at random and hidden from the user, so the user cannot choose which model gets which prompt. We did not add any private or personal data of our own.

2.2 Building the dataset

All of the cleaning happens in `make_data.py`. Table 1 shows how many rows survive each step, and the paragraphs after it explain why we did each one.

Step	What is left
Raw dataset	33,000 battles
English only, first exchange, 8 models kept, 10 to 800 words	27,216 rows
Remove exact duplicate responses within a family	26,925 rows
Downsample every family to the smallest one (2,383 each)	11,915 rows

Table 1: Data pipeline. Each battle can give up to two rows, one per model.

First we keep only battles whose language is English, and from each conversation we take only the first user message and the first assistant reply. Multi-turn conversations are still useful this way,

since their first exchange is a normal prompt and response pair. Each battle gives two rows, one for model A and one for model B. Next we keep eight models and group them into five families (Table 2).

Family	Models kept	Rows after filtering
Vicuna	vicuna-13b (4,945), vicuna-7b (2,417)	7,362
GPT	gpt-3.5-turbo (3,834), gpt-4 (3,463)	7,297
Claude	claude-v1 (3,272), claude-instant-v1 (2,168)	5,440
Koala	koala-13b (4,559)	4,559
PaLM	palm-2 (2,558)	2,558
Total		27,216

Table 2: Models we kept and how many rows each one had after the English and length filters.

GPT, Claude and PaLM come from three different companies, so they easily satisfy the "at least three families" requirement. Vicuna and Koala are a bit special because both are fine-tuned versions of Meta's LLaMA, made by two different research groups [4, 5]. We kept them as separate classes on purpose. It makes the task harder, but it also lets us ask whether two models with the same base model can still be told apart, which turned out to be one of the more interesting parts of the results.

We dropped responses shorter than 10 words or longer than 800 words. Very short answers like "Yes." carry almost no style, and the very long ones would get cut at 300 tokens anyway. Then we removed exact duplicate responses within each family. This removed 291 rows, and 175 of them were from PaLM. We did not go through all of them by hand, but our guess is that most are repeated canned answers, since the same identical text showing up again and again from one model is hard to explain any other way.

After that, PaLM was the smallest class with 2,383 rows, so we randomly downsampled every family to 2,383. That gives 11,915 rows in total, perfectly balanced. We would rather throw away data than let the classifier learn "when in doubt, say Vicuna", which is what tends to happen when one class is three times bigger than another.

Each row is saved to a CSV file with the columns in Table 3.

Column	Meaning
`qid`	Arena battle id, shared by the two rows from the same battle
`model`	exact model name, e.g. `gpt-3.5-turbo`
`family`	our class label (Claude, GPT, Koala, PaLM or Vicuna)
`prompt`	first user message (x_{in})
`response`	first reply from this model (x_{out})
`task`	keyword-based task label used for RQ3 (Section 2.4)

Table 3: Columns of `data/train.csv`, `val.csv` and `test.csv`.

2.3 Splitting by question, not by row

This is the part of the data pipeline we were most careful about. Because every battle shows the same prompt to two models, the same prompt appears in two rows with two different labels. If we

split the rows randomly, a prompt could land in the training set with the label "GPT" and in the test set with the label "Claude". That would make the input-only experiment in RQ2 meaningless and it would let the input+output model learn prompt-specific shortcuts. So we split by `qid` instead: all rows from one battle go to the same split. We used 70% of the battles for training, 15% for validation and 15% for testing, which gave 8,368 training rows, 1,784 validation rows and 1,763 test rows.

Since whole battles are assigned to a split, the classes inside each split are only roughly balanced. The test set has 335 Claude, 334 GPT, 357 Koala, 352 PaLM and 385 Vicuna examples, which is close enough that accuracy and macro F1 end up very similar.

2.4 Task labels for RQ3

The Arena data does not say what kind of task a prompt is, so for RQ3 we labeled prompts with a simple keyword rule. A prompt that mentions things like "python", "function", "sql" or a code block is *coding*. One that asks for a poem, story, essay, letter or lyrics is *writing*. One with words like "solve", "calculate" or "probability" is *math*. Everything else is *general*. This gave 9,207 general, 1,535 coding, 777 math and 396 writing prompts. The rule is obviously rough (a prompt asking for a story about a programmer counts as coding because "program" is checked first), and we come back to this when we discuss RQ3.

3 Models and Training

3.1 Preprocessing

We wrote our own tokenizer instead of using a pretrained one. Text is lowercased and split with a regular expression that keeps runs of letters and digits as words and turns every punctuation mark into its own token. So `**Note:**` becomes `* * note : * *`. We did this because we expected formatting (bold, bullets, colons) to be part of each model's style, and a tokenizer that throws punctuation away would hide that. For the same reason, newlines are replaced with a special token `nwln` before splitting, so the model can see line breaks and list structure.

The vocabulary is the 20,000 most frequent tokens in the training set, plus a padding token and an unknown token (20,002 total). It is built from the training split only, so no test words leak in. Every example is cut or padded to 300 tokens. For the input+output setting we join the two with a separator token, `prompt sepsep response`, and cut the result at the same 300 tokens so that all three settings see inputs of the same size.

We did not use pretrained word embeddings. The embeddings are learned from scratch, mostly because we wanted to see what can be learned from this data alone, and because tokens like `*` or `nwln` do not have useful pretrained vectors anyway.

Figure 1 shows both models side by side, and Table 4 lists every setting.

3.2 CNN

Our CNN follows the sentence classification CNN from Kim [2]. Each token is mapped to a 100-dimensional embedding. Three 1D convolution layers with filter widths 3, 4 and 5 (100 filters each) slide over the sequence, so they act like detectors for 3, 4 and 5-token patterns. After a ReLU, each filter is max-pooled over the whole sequence, which keeps only the strongest match for that

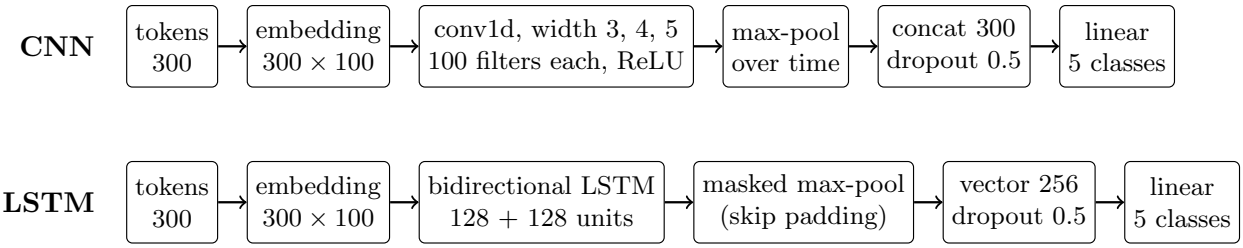


Figure 1: The two models. They share the same input pipeline and output layer and only differ in the middle.

pattern anywhere in the text. The three pooled vectors are concatenated into a 300-dimensional vector, passed through dropout with $p = 0.5$, and a linear layer maps it to the 5 classes. The model has 2,122,005 parameters, and 2,000,200 of them are the embedding table.

One nice thing about this design for our problem is that max pooling does not care where in the answer a pattern shows up. A pattern like `nwln - * *` (a bullet point that starts with bold text) counts the same whether it is in the first line or the tenth.

3.3 LSTM

For the RNN we used a bidirectional LSTM [3] with 128 hidden units in each direction, on top of the same kind of 100-dimensional embedding. The usual way to get a sentence vector out of an LSTM is to take the last hidden state, but our inputs are right-padded to 300 tokens, and a lot of responses are much shorter than that. For a 120-token answer the last hidden state would have spent its last 180 steps reading padding. So instead we take the LSTM output at every position, mask out the padding positions, and max-pool over the real tokens only. The 256-dimensional result goes through dropout ($p = 0.5$) and a linear layer to the 5 classes. This model has 2,237,005 parameters, so the two models are almost the same size and the comparison between them is fair.

3.4 Training setup

Both models are trained the same way. For a batch of N examples, the model outputs a score $z_{i,c}$ for each class, and we minimize the cross-entropy loss

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^{N} \log \frac{\exp(z_{i,y_i})}{\sum_{c=1}^{5} \exp(z_{i,c})}.$$

After every epoch we compute accuracy on the validation set and save a copy of the weights whenever it is the best so far. At the end we reload the best copy and evaluate it once on the test set. The test set is never looked at during training or model selection.

We kept these hyperparameters the same across every experiment in the report (all six RQ1/RQ2 runs, RQ3 and the RQ4 ablations), so differences between experiments come from the input data and not from tuning. We did not have time for a real hyperparameter search, which is one of the things we would do with more time.

Figure 2 shows the validation accuracy per epoch. Keeping the best epoch turned out to matter for the LSTM: on output only, it peaked at 64.1% in epoch 7 and then dropped to 60.9% in epoch 8, even though its training loss kept going down (0.719 to 0.648). That is the start of overfitting, and without the checkpoint we would have reported the worse model.

Setting	Value
Vocabulary	20,000 most frequent training tokens + `<pad>` + `<unk>`
Max length	300 tokens (cut or padded)
Embedding	100 dimensions, trained from scratch
CNN	filter widths 3, 4, 5, 100 filters each, ReLU, max-pool
LSTM	1 layer, bidirectional, 128 units per direction, masked max-pool
Dropout	0.5 before the output layer
Loss / optimizer	cross-entropy / Adam, learning rate 0.001
Batch size / epochs	64 / 8
Model selection	best validation accuracy over the 8 epochs
Random seed	42
Hardware	Google Colab, T4 GPU

Table 4: Hyperparameters. They are the same for every experiment in this report.

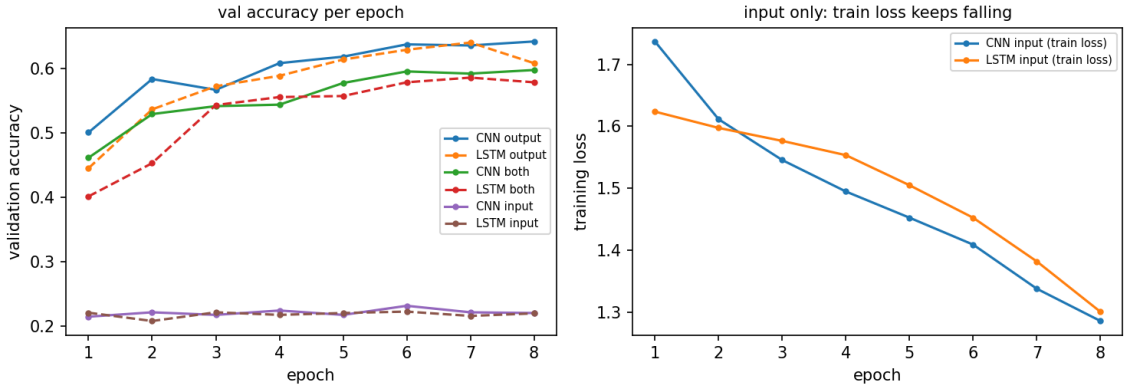


Figure 2: Left: validation accuracy after each epoch for all six RQ1/RQ2 runs. Right: training loss for the two input-only runs.

4 Results

4.1 RQ1: Can we identify which LLM generated a response?

Yes, much better than chance. Using only the response text, the CNN gets 61.2% test accuracy and the LSTM gets 61.7%, against a 20% random baseline (Table 5).

The overall accuracy hides a big difference between classes, though. Table 6 has the per-class F1 scores, and the middle column of Figure 3 has the confusion matrices.

Claude is by far the easiest class. Both models get an F1 near 0.88, and almost nothing else gets mistaken for Claude (the Claude column is nearly empty in both output-only matrices). PaLM comes second at around 0.70. The other three classes are where things fall apart. Koala is the worst: the CNN only recognizes 30% of Koala answers, and it labels 36% of them as Vicuna. Vicuna and GPT also get confused with each other in both directions, with roughly 20% of each going to the other.

This pattern makes sense once you look at where these models come from. Vicuna was trained by fine-tuning LLaMA on about 70K ChatGPT conversations that users shared on ShareGPT [4], so

Model	Input used	Best epoch	Val acc	Test acc	Test macro F1
CNN	output only	8	0.642	0.612	0.605
LSTM	output only	7	0.641	**0.617**	**0.623**
CNN	input only	6	0.232	0.205	0.197
LSTM	input only	6	0.223	0.212	0.210
CNN	input + output	8	0.598	0.577	0.563
LSTM	input + output	7	0.586	0.580	0.580
Random guess			0.200	0.200	0.200

Table 5: Main results on the 1,763-example test set. The first two rows answer RQ1, and the whole table answers RQ2.

Model	Claude	GPT	Koala	PaLM	Vicuna
CNN (output only)	0.874	0.549	0.417	0.699	0.483
LSTM (output only)	0.885	0.573	0.478	0.722	0.456

Table 6: Per-class F1 on the test set, output only.

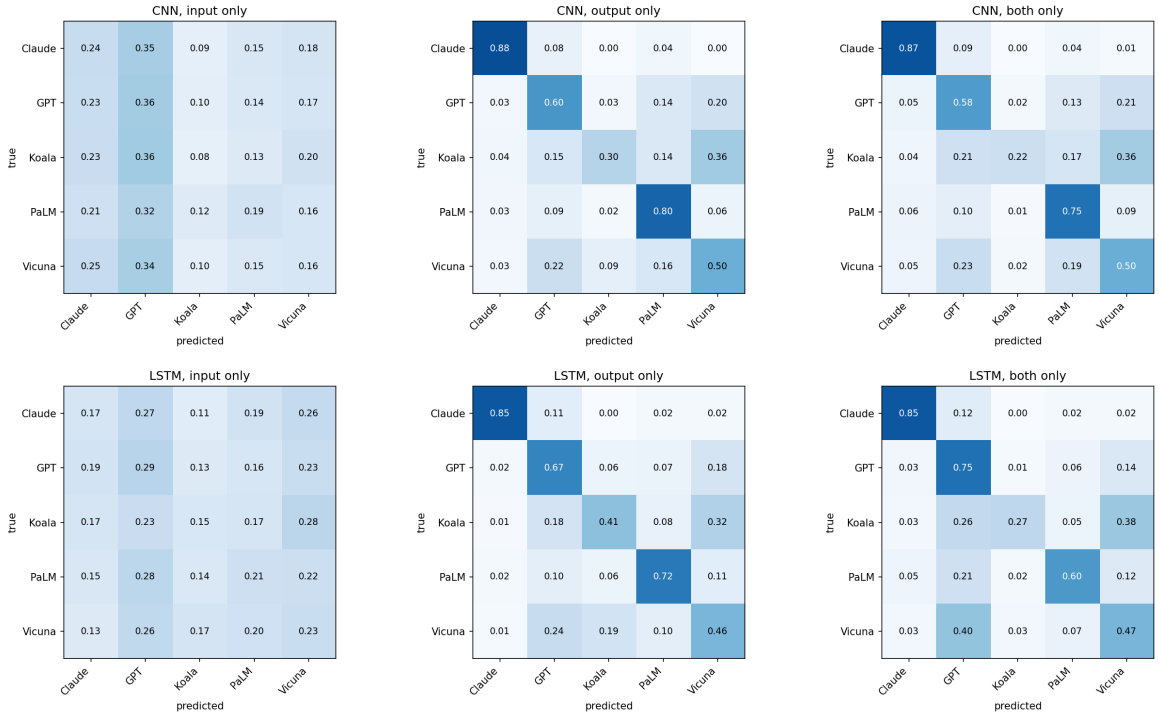


Figure 3: Confusion matrices on the test set. Top row is the CNN, bottom row is the LSTM. Columns are input only, output only, and input + output. Each row of a matrix is normalized, so it shows where the examples of one true class ended up.

it was literally trained to imitate GPT's answers. Koala is also a LLaMA fine-tune, and its training data includes ShareGPT conversations as well [5]. So three of our five classes are, in a sense, GPT and two students of GPT. Claude and PaLM were trained completely separately and they stand out much more. We think this is the most interesting finding of RQ1: the classifier does best on models with a truly separate training history, and worst on models that learned from each other.

CNN versus LSTM is basically a tie. The LSTM is 0.5 points higher in accuracy and 1.8 points higher in macro F1, mostly because it finds more of the Koala answers (recall 0.415 versus 0.303). But as we show in Section 5.3, just re-running the same CNN gives a difference bigger than 0.5 points, so we would not claim either model is better from these numbers. If we had to pick one for this task we would pick the CNN, simply because it is the simpler model and does not need the padding trick that the LSTM needs.

4.2 RQ2: Does the user's prompt help identify the LLM?

No. In our data the prompt on its own carries no information about the model, and adding it to the response makes things slightly worse (Figure 4).

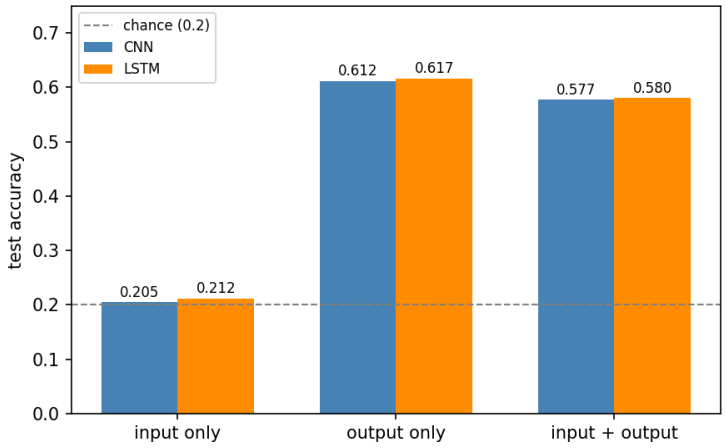


Figure 4: Test accuracy for the three input settings. The dashed line is random guessing.

Input only. Both models land right at chance: 20.5% for the CNN and 21.2% for the LSTM. Their confusion matrices (left column of Figure 3) are flat, with every row looking about the same. The CNN basically fell back to guessing GPT about a third of the time no matter what the true class was. The training curves tell the same story (Figure 2, right). The CNN's training loss went from 1.74 down to 1.29, which is well below $\ln 5 \approx 1.61$, the loss of a uniform guess. So it was clearly learning something about the training prompts, while validation accuracy stayed stuck around 22% the whole time. That is pure memorization: it learned which training prompts went with which label, and none of it carried over to new prompts.

This is actually what we should expect from Chatbot Arena. The two models in each battle are chosen at random and hidden from the user, so the prompt is independent of the label by design. There is nothing to learn. The project asks whether the LLM can be predicted from the prompt even before seeing its response. For this dataset the answer is no, and the reason is how the data was collected, not a weakness of the classifiers.

The project also asks why input-only classification might work "surprisingly well" in other setups. We think it would happen any time the prompts were not randomly assigned. For example, if a team built its dataset by asking GPT a set of coding questions and asking Claude a set of writing questions, or by merging datasets from different sources, then the prompt would tell you which source the row came from, and the classifier would learn the dataset instead of the model. A high input-only number is a warning sign about the data more than a finding about the LLMs. Splitting by battle also matters here, as explained in Section 2.3.

Input + output. Adding the prompt to the response lowered accuracy by 3.4 points for the CNN (61.2% to 57.7%) and 3.6 points for the LSTM (61.7% to 58.0%). We see two reasons. First, the prompt takes up part of the fixed 300-token window, so for long prompts the end of the response gets cut off and the model sees less of the part that actually matters. Second, we already saw that these models will happily memorize prompt words that do not generalize, so the prompt adds a source of overfitting. Koala suffered the most here, with its recall dropping from 0.303 to 0.216 for the CNN and from 0.415 to 0.275 for the LSTM. Koala was already the hardest class, so any extra noise pushes it over the edge. Otherwise the right column of Figure 3 looks like a slightly blurrier copy of the middle column.

5 Further Analysis

5.1 RQ3 (extra credit): Do fingerprints generalize across tasks?

For this experiment we put all 11,915 rows back together and split them again by task. The classifiers train on general, coding and math prompts only (8,654 train and 1,149 validation rows, again split by battle). We then test them twice: on 1,716 held-out rows from those same three tasks ("in-domain"), and on all 396 writing rows, a kind of task the model never saw during training. We used output only here, since RQ2 showed the prompt does not help.

	In-domain (1,716)		Writing, unseen (396)	
Model	Acc	Macro F1	Acc	Macro F1
CNN	0.608	0.599	0.548	0.511
LSTM	0.611	0.603	0.538	0.492

Table 7: RQ3 results. Training uses general, coding and math prompts only.

Does performance drop on the unseen task? Yes, but less than we expected. Accuracy drops 6.0 points for the CNN and 7.3 points for the LSTM. The macro F1 drops more (8.8 and 11.1 points), which means some classes held up fine and others fell apart. Both models are still way above the 20% baseline on writing, so at least part of what they learned is a real fingerprint and not just task-specific patterns.

Are some models more consistent across tasks? Very much. Claude was recognized 98.7% of the time on writing prompts by both models, which is even higher than its recall in RQ1 (0.88 and 0.85). But its precision on writing is only 0.58 for the CNN and 0.66 for the LSTM, because between 11 and 20% of every other model's writing also got labeled as Claude. Our reading is that "long, structured, creative answer" became associated with Claude, so the model over-predicts Claude when the task itself pushes every model toward that kind of answer. PaLM also held up

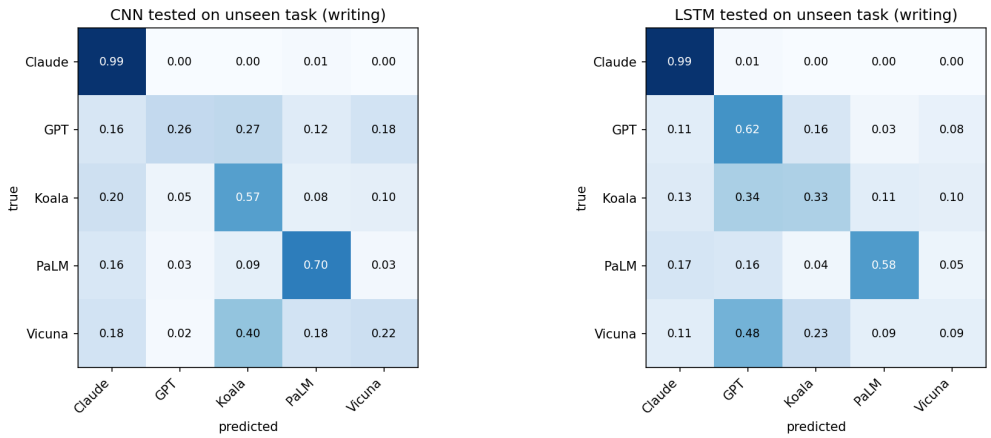


Figure 5: Confusion matrices on the unseen writing task.

reasonably well (recall 0.70 and 0.58). Vicuna nearly disappeared, with recall 0.215 for the CNN and only 0.092 for the LSTM. GPT is the strangest case: its recall was 0.26 with the CNN but 0.62 with the LSTM. Two models trained on the same data disagreeing that much tells us GPT's writing fingerprint is not stable, and with only 92 GPT writing examples we would not read much into either number.

General fingerprint or task-specific patterns? Our honest answer is both, and it depends on the model. Claude and PaLM seem to carry a fingerprint that survives a change of task. For Vicuna and GPT, it looks much more like the classifier learned how they answer general questions in particular, and that does not transfer.

There are two caveats we want to be upfront about. The writing set is small (65 to 92 examples per class), so a handful of examples can move a per-class number by several points. And our task labels come from a keyword rule, so some "writing" prompts are probably not really writing tasks, and some real writing prompts ended up labeled as general. The training data is also dominated by general prompts (9,207 of 11,915 rows overall), so "trained on three tasks" really means "trained mostly on general questions".

5.2 RQ4 (extra credit): What characteristics distinguish the LLMs?

To understand why the classifiers work, we first measured some simple surface features on the training responses (Table 8, Figures 6 and 7).

A few things jump out. PaLM uses markdown bold (**) about 1.5 times per answer, while every other model almost never does (0.04 or less). That alone is close to a PaLM detector, and it probably explains a good part of why PaLM is the second easiest class. PaLM also writes the longest answers with the shortest words. Claude is the bullet point model, with 2.6 bullet lines per answer compared to under 0.5 for GPT, Koala and Vicuna, and its length histogram has a clear bump around 230 to 290 words that no other model has. GPT gives the shortest answers on average, and its histogram has a big spike under 50 words, so it often answers briefly. GPT also has the highest type-token ratio, but we should be careful there: type-token ratio naturally goes up when a text is shorter, so part of that is just the length difference showing up again. Koala writes the longest sentences.

Family	Words	Word len	Words/sent.	TTR	Bullets	Numbered	Bold
Claude	177.5	5.01	18.8	0.643	**2.62**	1.38	0.008
GPT	137.2	5.03	18.2	**0.697**	0.46	**1.89**	0.029
Koala	157.7	5.02	**22.0**	0.589	0.50	1.66	0.009
PaLM	**188.3**	4.70	17.0	0.557	2.17	0.68	**1.539**
Vicuna	144.7	4.90	20.1	0.626	0.42	1.63	0.039

Table 8: Average surface features per response on the training set. Word len is the average word length in characters, TTR is the type-token ratio (unique words divided by total words), and Bullets, Numbered and Bold count bullet lines, numbered list lines and ** markers per response. Punctuation per 100 words was between 20 and 23 for everyone, so we left it out.

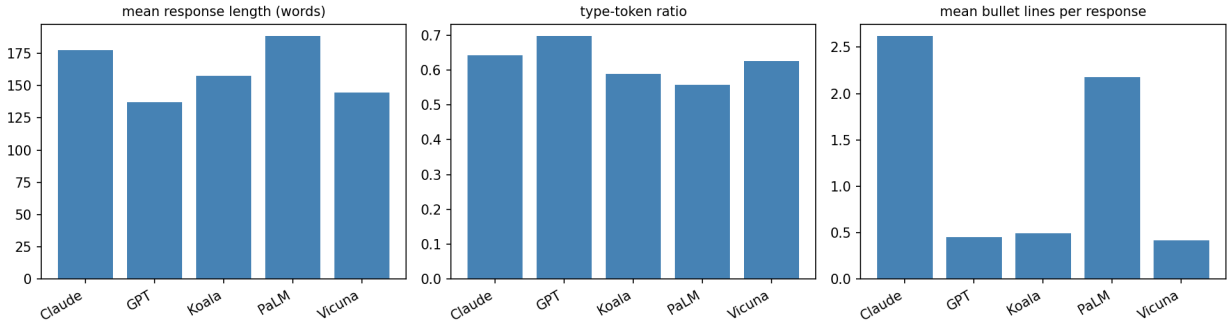


Figure 6: Three of the features from Table 8.

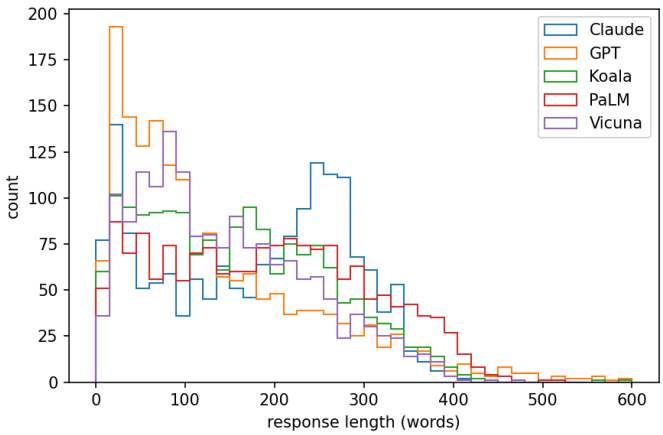


Figure 7: Distribution of response length (in words) for each family.

GPT, Koala and Vicuna also look very similar on almost every column. That lines up nicely with the RQ1 confusion matrices, where those three are the ones that get mixed up.

How far do these features get you? We trained a logistic regression on just 11 of these hand-made features (standardized using the training set). It reaches 40.3% accuracy, double the random baseline, but still about 20 points below the CNN. So the simple statistics explain a real part of the fingerprint, but the neural models are clearly using more than length and bullet counts.

Ablations. To see which signals the CNN relies on, we retrained it on modified versions of the responses and measured how much accuracy dropped. We used the CNN because a convolution looks at all positions at once and is cheaper to train than an LSTM, which has to step through the sequence one token at a time. All settings are the same as in RQ1.

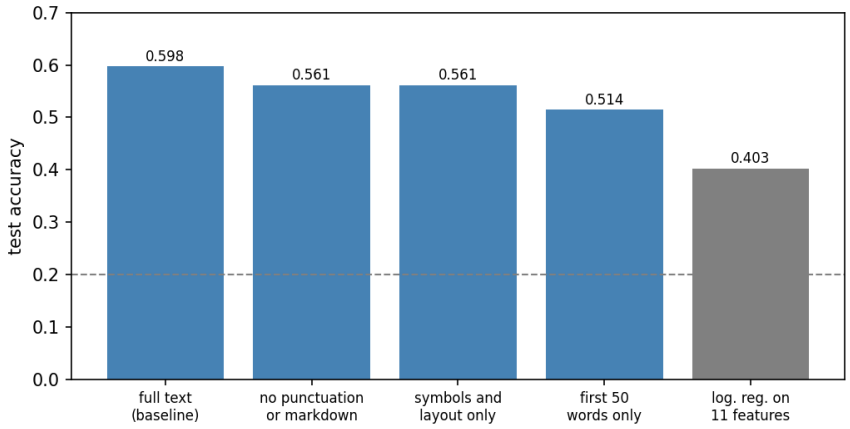


Figure 8: CNN test accuracy when parts of the response are removed. The gray bar is the logistic regression on hand-made features. The dashed line is random guessing.

What the CNN sees	Test acc	Macro F1	Change in acc
Full response (rerun of baseline)	0.598	0.587	
Words only (all symbols and newlines removed)	0.561	0.539	−3.6
Symbols and layout only (letters, digits removed)	0.561	0.564	−3.6
First 50 words only	0.514	0.503	−8.3
Logistic regression, 11 hand features	0.403	0.395	

Table 9: RQ4 ablation results.

The result that surprised us the most is the "symbols and layout only" row. In that setting we deleted every single letter and digit, so the CNN only saw things like `. , * : nwln - ( )`. It still got 56.1%, and its macro F1 (0.564) was actually higher than the words-only version. In other words, you can identify the model fairly well without reading a single word of what it said, just from how it punctuates and lays out its answer.

Going the other way, removing all punctuation and formatting and keeping only the words also gives 56.1%. So each type of signal alone gets most of the way to the full result, and together they add about 3.6 more points. The signals are partly redundant, which makes sense since a model's word choice and its formatting habits both come from the same training.

Cutting the response down to its first 50 words caused the biggest drop (8.3 points). This removes the length information completely, and it also throws away most of the answer, including whatever each model tends to do at the end of a response. The first 50 words alone are still worth 51.4%, so the beginning of an answer already gives a lot away.

So to answer the RQ4 questions directly: the classifier uses both linguistic and structural signals. Length, bullet lists and bold markdown are the clearest structural signals, and words carry about

the same amount of information. Removing any one kind of signal hurts, but not a lot, because the other kind covers for it. Removing length and most of the content hurts the most.

5.3 How much of this is noise?

While writing this up we noticed that the "full text" baseline in the RQ4 ablation is exactly the same experiment as the CNN output-only run in RQ1: same code, same data, same seed. But one got 61.2% and the other got 59.8%. Looking at the logs, the two runs are identical for the first three epochs and then start to drift apart in the fourth decimal of the loss in epoch 4 (1.0749 versus 1.0750). The cause is that some GPU operations in PyTorch are not deterministic by default, so even with a fixed seed two runs are not exactly the same, and small differences grow over training.

This gives us a rough sense of the noise: about 1.4 points of accuracy between two "identical" runs. That changes how we read some of our results. The CNN versus LSTM gap in RQ1 (0.5 points) is well inside this noise, so we treat them as equal. The 3.4 to 3.6 point drop from adding the prompt in RQ2 and the 3.6 point drops in the ablations are bigger than this noise, but not by a huge margin, so we would want 3 to 5 runs with different seeds before making strong claims about them. The 8.3 point drop from truncation and the jump from 20% to about 61% are far outside the noise and we are confident about those.

6 Lessons Learned

The biggest lesson for us is that the data decisions mattered more than the model decisions. Two models with very different architectures ended up within half a point of each other, while choices like splitting by battle instead of by row, or including the prompt, changed what our numbers even meant. If we had split by row, the input-only experiment would have been comparing apples and oranges and we might not have noticed.

We also learned to read a result of "20%" correctly. When the input-only models came back at chance it is tempting to think something is broken. But once we thought about how Chatbot Arena assigns models to prompts, chance was the right answer, and the training curves (loss going down, validation flat) confirmed that the models were memorizing and not learning. A result that matches what the data collection process predicts is a good result even when the number is low.

Overall accuracy alone did not tell the story. 61% sounds mediocre, but the confusion matrices show it is really two very easy classes, Claude and PaLM, plus three classes that are hard to separate because they are closely related. Without the per-class breakdown we would have missed the most interesting finding of the project, which is that the models trained to imitate ChatGPT are the hardest to tell apart from ChatGPT.

On the practical side, we learned that a fixed random seed does not guarantee identical results on a GPU, and that one run per setting is not enough when differences are small. If we did this again we would run every setting with several seeds from the start and report the mean and spread. Saving the best validation checkpoint also turned out to be worth it, since the LSTM started overfitting in its last epoch. A smaller lesson: our training script only printed the per-epoch numbers to the screen and never saved them, so to make Figure 2 we had to copy them out of the Colab output by hand. Next time every number we might want to plot goes straight into a file.

If we had more time, there are a few things we would like to try:

- multiple seeds for every experiment, as discussed above;

- pretrained embeddings or a small pretrained transformer, to see how much higher accuracy can go;
- better task labels for RQ3, either by labeling a sample by hand or by training a task classifier, and a larger writing set;
- newer models. Our data is from 2023, so it covers GPT-3.5/4, Claude v1 and PaLM 2. Models have changed a lot since then and a fingerprint learned on old versions probably does not transfer to new ones;
- a length-matched comparison, where we only compare responses of similar length, to see how much accuracy is left once the easiest signal is controlled for.

Finally, a note on limitations. 61% accuracy is far above chance but far from reliable. Even our best class, Claude, has a precision that drops to 0.58 in some settings. A tool like ours should not be used to make claims about who or what wrote a particular piece of text, especially since the fingerprints likely change every time a model is updated.

References

[1] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *NeurIPS Datasets and Benchmarks Track*, 2023.

[2] Y. Kim. Convolutional neural networks for sentence classification. In *Proceedings of EMNLP*, 2014.

[3] S. Hochreiter and J. Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735-1780, 1997.

[4] W.-L. Chiang, Z. Li, Z. Lin, Y. Sheng, Z. Wu, H. Zhang, L. Zheng, S. Zhuang, Y. Zhuang, J. E. Gonzalez, I. Stoica, and E. P. Xing. Vicuna: An open-source chatbot impressing GPT-4 with 90% ChatGPT quality. LMSYS blog, 2023.

[5] X. Geng, A. Gudibande, H. Liu, E. Wallace, P. Abbeel, S. Levine, and D. Song. Koala: A dialogue model for academic research. BAIR blog, 2023.

A How to Reproduce Our Results

Everything runs on a free Google Colab GPU. You need a HuggingFace account that has accepted the terms of `lmsys/chatbot_arena_conversations`, and its access token stored in the `HF_TOKEN` environment variable. Then, from the repository folder:

```
pip install torch datasets pandas scikit-learn matplotlib
python make_data.py      # download, clean, balance, split into data/
python train.py all      # RQ1 and RQ2, six runs
python rq3.py            # RQ3
python rq4.py            # RQ4 features, logistic regression, ablations
python report_plots.py   # extra figures used in this report
```

All accuracy numbers are appended to `results/summary.txt`, per-class reports are saved as text files, and every figure in this report is saved as a PNG in `results/`.